

**Продвинутые стохастические градиентные
методы. Методы редукции дисперсии.
Muon**

Daniil Merkulov

Optimization methods. MIPT

Задача конечной суммы

Задача конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск работает следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .

Задача конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск работает следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .
- Сходимость с постоянным α или линейным поиском.

Задача конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск работает следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .
- Сходимость с постоянным α или линейным поиском.

Задача конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск работает следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .
- Сходимость с постоянным α или линейным поиском.

Перейдём от вычисления полного градиента к его несмещённой оценке. Случайно выбираем индекс i_k на каждой итерации равномерно:

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k) \quad (\text{SGD})$$

При $p(i_k = i) = \frac{1}{n}$ стохастический градиент является несмещённой оценкой градиента:

$$\mathbb{E}[\nabla f_{i_k}(x)] = \sum_{i=1}^n \frac{1}{n} \nabla f_i(x) = \nabla f(x)$$

Результаты для GD и SGD

Стохастические итерации в n раз дешевле, но сколько итераций нужно?

Если ∇f — липшицев, то:

Предположение	Градиентный спуск	Стохастический ГС
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- SGD — дешёвая итерация, но медленная сходимость.

Результаты для GD и SGD

Стохастические итерации в n раз дешевле, но сколько итераций нужно?

Если ∇f — липшицев, то:

Предположение	Градиентный спуск	Стохастический ГС
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- SGD — дешёвая итерация, но медленная сходимость.
 - Сублинейная скорость даже в сильно выпуклом случае.

Результаты для GD и SGD

Стохастические итерации в n раз дешевле, но сколько итераций нужно?

Если ∇f — липшицев, то:

Предположение	Градиентный спуск	Стохастический ГС
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- SGD — дешёвая итерация, но медленная сходимость.
 - Сублинейная скорость даже в сильно выпуклом случае.
 - Оценки неулучшаемы при стандартных предположениях.

Результаты для GD и SGD

Стохастические итерации в n раз дешевле, но сколько итераций нужно?

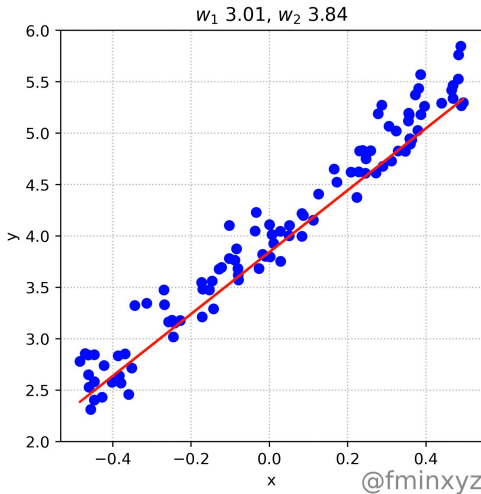
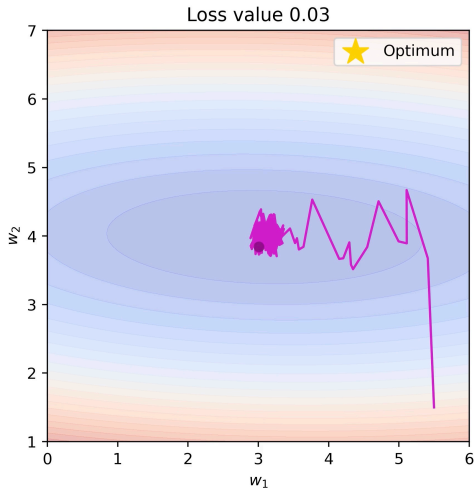
Если ∇f — липшицев, то:

Предположение	Градиентный спуск	Стохастический ГС
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклая	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- SGD — дешёвая итерация, но медленная сходимость.
 - Сублинейная скорость даже в сильно выпуклом случае.
 - Оценки неулучшаемы при стандартных предположениях.
- Моментум и квазиньютоновские методы **не улучшают скорость** в стохастическом случае. Могут улучшить только константы (бутылочное горлышко — дисперсия, а не число обусловленности).

SGD с постоянным шагом не сходится

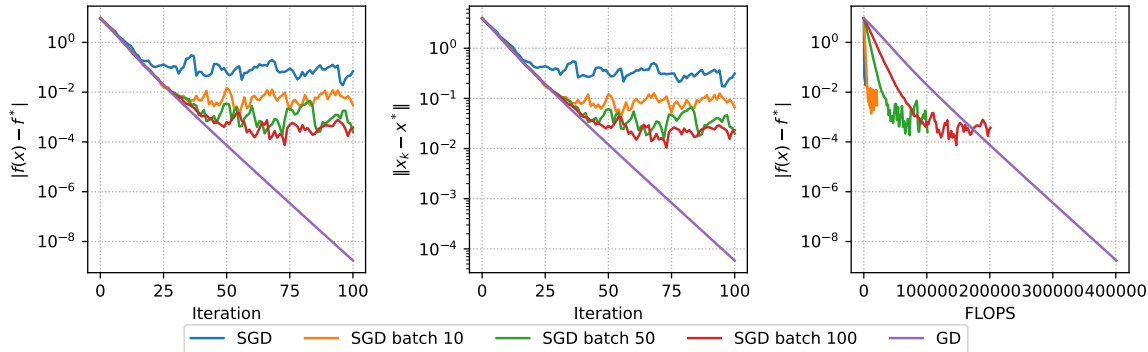
Stochastic Gradient Descent. Batch = 2



Основная проблема SGD

$$f(x) = \frac{\mu}{2} \|x\|_2^2 + \frac{1}{m} \sum_{i=1}^m \log(1 + \exp(-y_i \langle a_i, x \rangle)) \rightarrow \min_{x \in \mathbb{R}^n}$$

Strongly convex binary logistic regression. $m=200$, $n=10$, $\mu=1$.



Методы редукции дисперсии

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения
 - Если $\alpha < 1$: возможно смещение (но меньше дисперсия).

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения
 - Если $\alpha < 1$: возможно смещение (но меньше дисперсия).
- Полезно, если Y положительно коррелирована с X .

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения
 - Если $\alpha < 1$: возможно смещение (но меньше дисперсия).
- Полезно, если Y положительно коррелирована с X .

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения
 - Если $\alpha < 1$: возможно смещение (но меньше дисперсия).
- Полезно, если Y положительно коррелирована с X .

Применение к оценке градиента:

- SVRG: Пусть $X = \nabla f_{i_k}(x^{(k-1)})$ и $Y = \nabla f_{i_k}(\tilde{x})$, при $\alpha = 1$ и сохранённой $\tilde{x}$.

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2(\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения
 - Если $\alpha < 1$: возможно смещение (но меньше дисперсия).
- Полезно, если Y положительно коррелирована с X .

Применение к оценке градиента:

- SVRG: Пусть $X = \nabla f_{i_k}(x^{(k-1)})$ и $Y = \nabla f_{i_k}(\tilde{x})$, при $\alpha = 1$ и сохранённой $\tilde{x}$.
- $\mathbb{E}[Y] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$ — полный градиент в $\tilde{x}$;

Ключевая идея редукции дисперсии

Принцип: снижение дисперсии выборки X с помощью другой случайной величины Y с известным математическим ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2(\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$
 - Если $\alpha = 1$: без смещения
 - Если $\alpha < 1$: возможно смещение (но меньше дисперсия).
- Полезно, если Y положительно коррелирована с X .

Применение к оценке градиента:

- SVRG: Пусть $X = \nabla f_{i_k}(x^{(k-1)})$ и $Y = \nabla f_{i_k}(\tilde{x})$, при $\alpha = 1$ и сохранённой $\tilde{x}$.
- $\mathbb{E}[Y] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$ — полный градиент в $\tilde{x}$;
- $X - Y = \nabla f_{i_k}(x^{(k-1)}) - \nabla f_{i_k}(\tilde{x})$

SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Поддерживаем таблицу градиентов g_i функции f_i , $i = 1, \dots, n$

SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Поддерживаем таблицу градиентов g_i функции f_i , $i = 1, \dots, n$
- Инициализируем $x^{(0)}$ и $g_i^{(0)} = \nabla f_i(x^{(0)})$, $i = 1, \dots, n$

SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Поддерживаем таблицу градиентов g_i функции f_i , $i = 1, \dots, n$
- Инициализируем $x^{(0)}$ и $g_i^{(0)} = \nabla f_i(x^{(0)})$, $i = 1, \dots, n$
- На шаге $k = 1, 2, 3, \dots$ выбираем случайный $i_k \in \{1, \dots, n\}$:

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{свежий градиент } f_{i_k})$$

Все остальные $g_i^{(k)} = g_i^{(k-1)}$, $i \neq i_k$ — не меняются

SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Поддерживаем таблицу градиентов g_i функции f_i , $i = 1, \dots, n$
- Инициализируем $x^{(0)}$ и $g_i^{(0)} = \nabla f_i(x^{(0)})$, $i = 1, \dots, n$
- На шаге $k = 1, 2, 3, \dots$ выбираем случайный $i_k \in \{1, \dots, n\}$:

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{свежий градиент } f_{i_k})$$

Все остальные $g_i^{(k)} = g_i^{(k-1)}$, $i \neq i_k$ — не меняются

- Обновление:

$$x^{(k)} = x^{(k-1)} - \alpha_k \frac{1}{n} \sum_{i=1}^n g_i^{(k)}$$

SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Поддерживаем таблицу градиентов g_i функции f_i , $i = 1, \dots, n$
- Инициализируем $x^{(0)}$ и $g_i^{(0)} = \nabla f_i(x^{(0)})$, $i = 1, \dots, n$
- На шаге $k = 1, 2, 3, \dots$ выбираем случайный $i_k \in \{1, \dots, n\}$:

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{свежий градиент } f_{i_k})$$

Все остальные $g_i^{(k)} = g_i^{(k-1)}$, $i \neq i_k$ — не меняются

- Обновление:

$$x^{(k)} = x^{(k-1)} - \alpha_k \frac{1}{n} \sum_{i=1}^n g_i^{(k)}$$

- Оценки градиента SAG **смещённые**, но с сильно сниженной дисперсией

SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Поддерживаем таблицу градиентов g_i функции f_i , $i = 1, \dots, n$
- Инициализируем $x^{(0)}$ и $g_i^{(0)} = \nabla f_i(x^{(0)})$, $i = 1, \dots, n$
- На шаге $k = 1, 2, 3, \dots$ выбираем случайный $i_k \in \{1, \dots, n\}$:

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{свежий градиент } f_{i_k})$$

Все остальные $g_i^{(k)} = g_i^{(k-1)}$, $i \neq i_k$ — не меняются

- Обновление:

$$x^{(k)} = x^{(k-1)} - \alpha_k \frac{1}{n} \sum_{i=1}^n g_i^{(k)}$$

- Оценки градиента SAG **смещённые**, но с сильно сниженной дисперсией
- Дорого ли усреднять все градиенты? Так же эффективно, как SGD, если считать аккуратно:

$$x^{(k)} = x^{(k-1)} - \alpha_k \underbrace{\left(\frac{1}{n} g_{i_k}^{(k)} - \frac{1}{n} g_{i_k}^{(k-1)} + \underbrace{\frac{1}{n} \sum_{i=1}^n g_i^{(k-1)}}_{\text{старое среднее}} \right)}_{\text{новое среднее}}$$

Сходимость SAG

Предположим $f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$, каждая f_i дифференцируема, ∇f_i — липшицев с константой L .

Обозначим $\bar{x}^{(k)} = \frac{1}{k} \sum_{l=0}^{k-1} x^{(l)}$ — среднее итератов после $k - 1$ шагов.

i Theorem

SAG с фиксированным шагом $\alpha = \frac{1}{16L}$ и инициализацией

$$g_i^{(0)} = \nabla f_i(x^{(0)}) - \nabla f(x^{(0)}), \quad i = 1, \dots, n$$

удовлетворяет

$$\mathbb{E}[f(\bar{x}^{(k)})] - f^* \leq \frac{48n}{k} [f(x^{(0)}) - f^*] + \frac{128L}{k} \|x^{(0)} - x^*\|^2$$

Сходимость SAG

- Результат для среднего итерата $\bar{x}^{(k)}$, но справедлив и для лучшего $x_{best}^{(k)}$.

Сходимость SAG

- Результат для среднего итерата $\bar{x}^{(k)}$, но справедлив и для лучшего $x_{best}^{(k)}$.
- Скорость $\mathcal{O}\left(\frac{1}{k}\right)$ для SAG. Сравните: $\mathcal{O}\left(\frac{1}{k}\right)$ для GD и $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$ для SGD.

Сходимость SAG

- Результат для среднего итерата $\bar{x}^{(k)}$, но справедлив и для лучшего $x_{best}^{(k)}$.
- Скорость $\mathcal{O}\left(\frac{1}{k}\right)$ для SAG. Сравните: $\mathcal{O}\left(\frac{1}{k}\right)$ для GD и $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$ для SGD.
- Но константы разные! После k шагов:

Сходимость SAG

- Результат для среднего итерата $\bar{x}^{(k)}$, но справедлив и для лучшего $x_{best}^{(k)}$.
- Скорость $\mathcal{O}\left(\frac{1}{k}\right)$ для SAG. Сравните: $\mathcal{O}\left(\frac{1}{k}\right)$ для GD и $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$ для SGD.
- Но константы разные! После k шагов:
 - GD: $\frac{L\|x^{(0)} - x^*\|^2}{2k}$

Сходимость SAG

- Результат для среднего итерата $\bar{x}^{(k)}$, но справедлив и для лучшего $x_{best}^{(k)}$.
- Скорость $\mathcal{O}\left(\frac{1}{k}\right)$ для SAG. Сравните: $\mathcal{O}\left(\frac{1}{k}\right)$ для GD и $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$ для SGD.
- Но константы разные! После k шагов:
 - GD: $\frac{L\|x^{(0)}-x^*\|^2}{2k}$
 - SAG: $\frac{48n[f(x^{(0)})-f^*]+128L\|x^{(0)}-x^*\|^2}{k}$

Сходимость SAG

- Результат для среднего итерата $\bar{x}^{(k)}$, но справедлив и для лучшего $x_{best}^{(k)}$.
- Скорость $\mathcal{O}\left(\frac{1}{k}\right)$ для SAG. Сравните: $\mathcal{O}\left(\frac{1}{k}\right)$ для GD и $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$ для SGD.
- Но константы разные! После k шагов:
 - GD: $\frac{L\|x^{(0)}-x^*\|^2}{2k}$
 - SAG: $\frac{48n[f(x^{(0)})-f^*]+128L\|x^{(0)}-x^*\|^2}{k}$
- Первый член SAG страдает от множителя n ; авторы рекомендуют хорошую инициализацию (например, n шагов SGD).

Сходимость SAG: сильно выпуклый случай

Дополнительно: каждая f_i сильно выпукла с параметром μ .

i Theorem

SAG с шагом $\alpha = \frac{1}{16L}$ и той же инициализацией:

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2} (f(x^{(0)}) - f^*) + \frac{4L}{n} \|x^{(0)} - x^*\|^2\right)$$

Замечания:

- Линейная скорость $\mathcal{O}(\gamma^k)$ для SAG. Сравните: $\mathcal{O}(\gamma^k)$ для GD и лишь $\mathcal{O}(\frac{1}{k})$ для SGD.

Сходимость SAG: сильно выпуклый случай

Дополнительно: каждая f_i сильно выпукла с параметром μ .

i Theorem

SAG с шагом $\alpha = \frac{1}{16L}$ и той же инициализацией:

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2} (f(x^{(0)}) - f^*) + \frac{4L}{n} \|x^{(0)} - x^*\|^2\right)$$

Замечания:

- Линейная скорость $\mathcal{O}(\gamma^k)$ для SAG. Сравните: $\mathcal{O}(\gamma^k)$ для GD и лишь $\mathcal{O}(\frac{1}{k})$ для SGD.
- Как и GD, SAG адаптивен к сильной выпуклости.

Сходимость SAG: сильно выпуклый случай

Дополнительно: каждая f_i сильно выпукла с параметром μ .

i Theorem

SAG с шагом $\alpha = \frac{1}{16L}$ и той же инициализацией:

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2} (f(x^{(0)}) - f^*) + \frac{4L}{n} \|x^{(0)} - x^*\|^2\right)$$

Замечания:

- Линейная скорость $\mathcal{O}(\gamma^k)$ для SAG. Сравните: $\mathcal{O}(\gamma^k)$ для GD и лишь $\mathcal{O}(\frac{1}{k})$ для SGD.
- Как и GD, SAG адаптивен к сильной выпуклости.
- Доказательство непростое: 15 страниц, с помощью компьютера!

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.
- На практике можно использовать стратегию бэктрекинга для оценки константы Липшица:

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.
- На практике можно использовать стратегию бэктрекинга для оценки константы Липшица:
 - Выбираем начальное L_0

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.
- На практике можно использовать стратегию бэктрекинга для оценки константы Липшица:
 - Выбираем начальное L_0
 - Увеличиваем L , пока не выполнится

$$f_{i_k}(x^{k+1}) \leq f_{i_k}(x^k) + \langle \nabla f_{i_k}(x^k), x^{k+1} - x^k \rangle + \frac{L}{2} \|x^{k+1} - x^k\|_2^2$$

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.
- На практике можно использовать стратегию бэктрекинга для оценки константы Липшица:
 - Выбираем начальное L_0
 - Увеличиваем L , пока не выполнится

$$f_{i_k}(x^{k+1}) \leq f_{i_k}(x^k) + \langle \nabla f_{i_k}(x^k), x^{k+1} - x^k \rangle + \frac{L}{2} \|x^{k+1} - x^k\|_2^2$$

- Уменьшаем L между итерациями

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.
- На практике можно использовать стратегию бэктрекинга для оценки константы Липшица:
 - Выбираем начальное L_0
 - Увеличиваем L , пока не выполнится

$$f_{i_k}(x^{k+1}) \leq f_{i_k}(x^k) + \langle \nabla f_{i_k}(x^k), x^{k+1} - x^k \rangle + \frac{L}{2} \|x^{k+1} - x^k\|_2^2$$

- Уменьшаем L между итерациями
- Поскольку стохастический градиент $g(x^k) \rightarrow \nabla f(x^k)$, можно использовать его норму для отслеживания сходимости (что неверно для SGD!)

Замечания к SAG

- Метод в ванильной формулировке не применим к большим нейросетям из-за требований к памяти.
- На практике можно использовать стратегию бэктрекинга для оценки константы Липшица:
 - Выбираем начальное L_0
 - Увеличиваем L , пока не выполнится

$$f_{i_k}(x^{k+1}) \leq f_{i_k}(x^k) + \langle \nabla f_{i_k}(x^k), x^{k+1} - x^k \rangle + \frac{L}{2} \|x^{k+1} - x^k\|_2^2$$

- Уменьшаем L между итерациями
- Поскольку стохастический градиент $g(x^k) \rightarrow \nabla f(x^k)$, можно использовать его норму для отслеживания сходимости (что неверно для SGD!)
- Для обобщённых линейных моделей (LogReg, МНК) достаточно хранить $\mathcal{O}(n)$ вместо $\mathcal{O}(pn)$:

$$f_i(w) = \varphi(w^T x_i) \quad \Rightarrow \quad \nabla f_i(w) = \varphi'(w^T x_i) x_i$$

SVRG (Stochastic Variance Reduced Gradient)

- Инициализация: $\tilde{x} \in \mathbb{R}^d$

SVRG (Stochastic Variance Reduced Gradient)

- Инициализация: $\tilde{x} \in \mathbb{R}^d$
- Для $i_{epoch} = 1$ до кол-ва эпох

SVRG (Stochastic Variance Reduced Gradient)

- Инициализация: $\tilde{x} \in \mathbb{R}^d$
- Для $i_{epoch} = 1$ до кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- **Для** $i_{epoch} = 1$ **до** кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$

SVRG (Stochastic Variance Reduced Gradient)

- Инициализация: $\tilde{x} \in \mathbb{R}^d$
- Для $i_{epoch} = 1$ до кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - Для $t = 1$ до длины эпохи (m)

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- **Для** $i_{epoch} = 1$ **до** кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - **Для** $t = 1$ **до** длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- **Для** $i_{epoch} = 1$ **до** кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - **Для** $t = 1$ **до** длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно
 - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- **Для** $i_{epoch} = 1$ **до** кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - **Для** $t = 1$ **до** длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно
 - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
 - Обновляем $\tilde{x} = x_m$

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- **Для** $i_{epoch} = 1$ **до** кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - **Для** $t = 1$ **до** длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно
 - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
 - Обновляем $\tilde{x} = x_m$

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- **Для** $i_{epoch} = 1$ **до** кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - **Для** $t = 1$ **до** длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно
 - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
 - Обновляем $\tilde{x} = x_m$

Замечания:

- Два вычисления градиента на внутреннем шаге.

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- Для $i_{epoch} = 1$ до кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - Для $t = 1$ до длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно
 - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
 - Обновляем $\tilde{x} = x_m$

Замечания:

- Два вычисления градиента на внутреннем шаге.
- Два параметра: длина эпохи + шаг α .

SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:** $\tilde{x} \in \mathbb{R}^d$
- Для $i_{epoch} = 1$ до кол-ва эпох
 - Вычисляем все градиенты $\nabla f_i(\tilde{x})$; сохраняем $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
 - Инициализируем $x_0 = \tilde{x}$
 - Для $t = 1$ до длины эпохи (m)
 - Выбираем $i_t \in \{1, \dots, n\}$ равномерно
 - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
 - Обновляем $\tilde{x} = x_m$

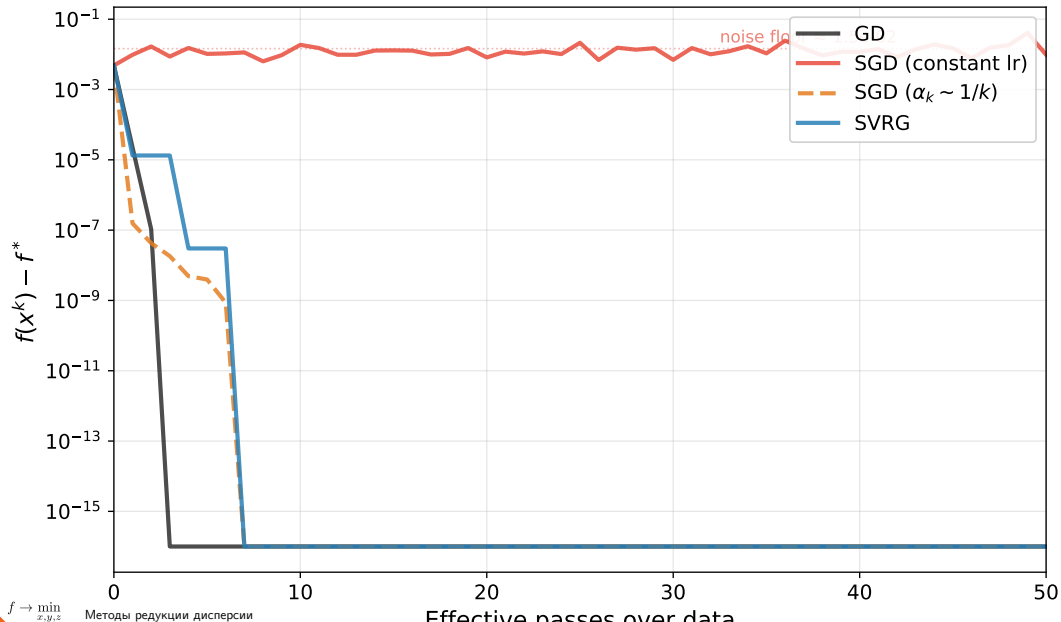
Замечания:

- Два вычисления градиента на внутреннем шаге.
- Два параметра: длина эпохи + шаг α .
- Линейная скорость сходимости, простое доказательство.

Сравнение методов

Метод	Стоимость итерации	Сходимость (сильно выпуклый)	Память
GD	$O(n)$	Линейная	$O(p)$
SGD	$O(1)$	Сублинейная $O(1/k)$	$O(p)$
SAG	$O(1)$	Линейная	$O(np)$
SVRG	$O(1)$	Линейная	$O(p)$

Эксперимент: SVRG vs SGD на логистической регрессии



Адаптивность и масштабирование

Adagrad (Duchi, Hazan, and Singer 2010)

Популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует настройки learning rate: $\alpha > 0$ — фиксированная константа, скорость обучения убывает автоматически.

Adagrad (Duchi, Hazan, and Singer 2010)

Популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует настройки learning rate: $\alpha > 0$ — фиксированная константа, скорость обучения убывает автоматически.
- Скорость обучения для редких информативных признаков убывает медленно.

Adagrad (Duchi, Hazan, and Singer 2010)

Популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует настройки learning rate: $\alpha > 0$ — фиксированная константа, скорость обучения убывает автоматически.
- Скорость обучения для редких информативных признаков убывает медленно.
- Значительно улучшает SGD в разреженных задачах.

Adagrad (Duchi, Hazan, and Singer 2010)

Популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует настройки learning rate: $\alpha > 0$ — фиксированная константа, скорость обучения убывает автоматически.
- Скорость обучения для редких информативных признаков убывает медленно.
- Значительно улучшает SGD в разреженных задачах.
- Главная слабость — монотонное накопление градиентов в знаменателе.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad: решает проблему монотонно убывающей скорости обучения. Использует экспоненциально затухающее среднее квадратов градиентов:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- Делит скорость обучения на скользящее среднее модулей последних градиентов.

RMSProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad: решает проблему монотонно убывающей скорости обучения. Использует экспоненциально затухающее среднее квадратов градиентов:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- Делит скорость обучения на скользящее среднее модулей последних градиентов.
- Позволяет более тонкую настройку, чем AdaGrad, подходит для нестационарных задач.

RMSProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad: решает проблему монотонно убывающей скорости обучения. Использует экспоненциально затухающее среднее квадратов градиентов:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- Делит скорость обучения на скользящее среднее модулей последних градиентов.
- Позволяет более тонкую настройку, чем AdaGrad, подходит для нестационарных задач.
- Широко используется при обучении рекуррентных нейросетей.

Adam (Kingma and Ba, 2014) ¹

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Корректирует смещение к нулю в начальных моментах.

Adam (Kingma and Ba, 2014) ¹

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Корректирует смещение к нулю в начальных моментах.
- Одна из самых цитируемых научных работ в мире

Adam (Kingma and Ba, 2014) ¹

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Корректирует смещение к нулю в начальных моментах.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной статье

Adam (Kingma and Ba, 2014) ¹

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Корректирует смещение к нулю в начальных моментах.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной статье
- Не сходится для некоторых простых задач (даже выпуклых)

Adam (Kingma and Ba, 2014) ¹

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Корректирует смещение к нулю в начальных моментах.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной статье
- Не сходится для некоторых простых задач (даже выпуклых)
- Почему-то очень хорошо работает для некоторых сложных задач

Adam (Kingma and Ba, 2014) ¹

Комбинирует элементы AdaGrad и RMSProp. Экспоненциально затухающее среднее градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$
$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Коррекция:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$
$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Корректирует смещение к нулю в начальных моментах.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной статье
- Не сходится для некоторых простых задач (даже выпуклых)
- Почему-то очень хорошо работает для некоторых сложных задач
- Гораздо лучше работает для языковых моделей, чем для задач компьютерного зрения — почему?

¹Adam: A Method for Stochastic Optimization

AdamW (Loshchilov & Hutter, 2017)

Решает проблему ℓ_2 -регуляризации в адаптивных оптимизаторах. Стандартная ℓ_2 -регуляризация добавляет $\lambda\|x\|^2$ к лоссу, порождая градиентный член λx . В Adam этот член масштабируется адаптивным $\text{lr} \left(\sqrt{\hat{v}_j} + \epsilon \right)$, связывая weight decay с магнитудами градиентов.

AdamW **развязывает** weight decay и адаптацию градиентов:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \left(\frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)$$

Замечания:

- Член weight decay $\lambda x_j^{(k-1)}$ добавляется *после* адаптивного градиентного шага.

AdamW (Loshchilov & Hutter, 2017)

Решает проблему ℓ_2 -регуляризации в адаптивных оптимизаторах. Стандартная ℓ_2 -регуляризация добавляет $\lambda\|x\|^2$ к лоссу, порождая градиентный член λx . В Adam этот член масштабируется адаптивным $\text{lr} \left(\sqrt{\hat{v}_j} + \epsilon \right)$, связывая weight decay с магнитудами градиентов.

AdamW **развязывает** weight decay и адаптацию градиентов:

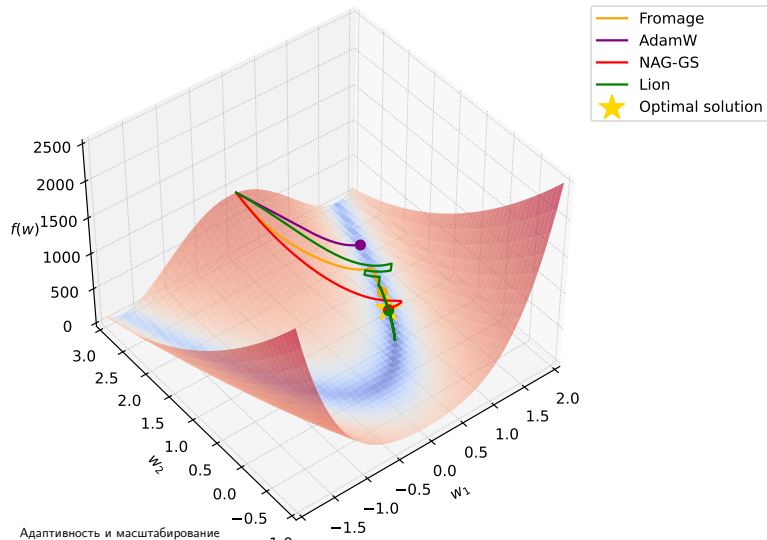
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \left(\frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)$$

Замечания:

- Член weight decay $\lambda x_j^{(k-1)}$ добавляется *после* адаптивного градиентного шага.
- Широко используется при обучении трансформеров и больших моделей. Выбор по умолчанию в HuggingFace Trainer.

Много оптимизаторов

Rosenbrock Function.
Adaptive stochastic gradient algorithms.
Learning rate 0.003



Как их сравнивать? AlgoPerf benchmark ²

- **AlgoPerf Benchmark:** Сравнивает алгоритмы обучения нейросетей по двум наборам правил:

Как их сравнивать? AlgoPerf benchmark ²

- **AlgoPerf Benchmark:** Сравнивает алгоритмы обучения нейросетей по двум наборам правил:
 - **External Tuning:** Моделирует подбор гиперпараметров с ограниченными ресурсами (5 попыток).

Как их сравнивать? AlgoPerf benchmark ²

- **AlgoPerf Benchmark:** Сравнивает алгоритмы обучения нейросетей по двум наборам правил:
 - **External Tuning:** Моделирует подбор гиперпараметров с ограниченными ресурсами (5 попыток).
 - **Self-Tuning:** Моделирует автоматическую настройку на одной машине (3× бюджет).

Как их сравнивать? AlgoPerf benchmark ²

- **AlgoPerf Benchmark:** Сравнивает алгоритмы обучения нейросетей по двум наборам правил:
 - **External Tuning:** Моделирует подбор гиперпараметров с ограниченными ресурсами (5 попыток).
 - **Self-Tuning:** Моделирует автоматическую настройку на одной машине (3× бюджет).
- **Оценка:** Агрегирует через профили производительности. Финальный балл: нормализованная площадь под профилем (1.0 = быстрееший на всех задачах).

Как их сравнивать? AlgoPerf benchmark ²

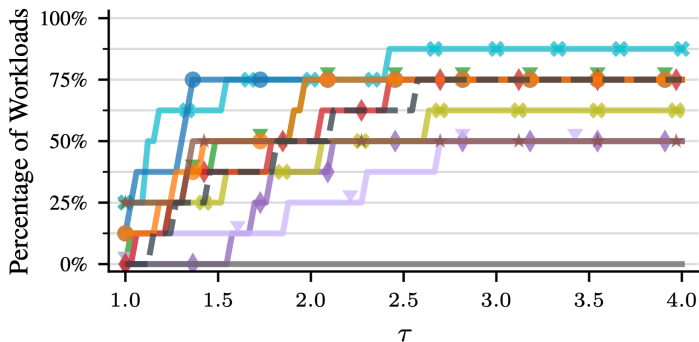
- **AlgoPerf Benchmark:** Сравнивает алгоритмы обучения нейросетей по двум наборам правил:
 - **External Tuning:** Моделирует подбор гиперпараметров с ограниченными ресурсами (5 попыток).
 - **Self-Tuning:** Моделирует автоматическую настройку на одной машине (3× бюджет).
- **Оценка:** Агрегирует через профили производительности. Финальный балл: нормализованная площадь под профилем (1.0 = быстрееший на всех задачах).
- **Стоимость:** Оценка потребовала $\sim 49,240$ часов на 8× NVIDIA V100 GPU.

²Benchmarking Neural Network Training Algorithms

AlgoPerf benchmark

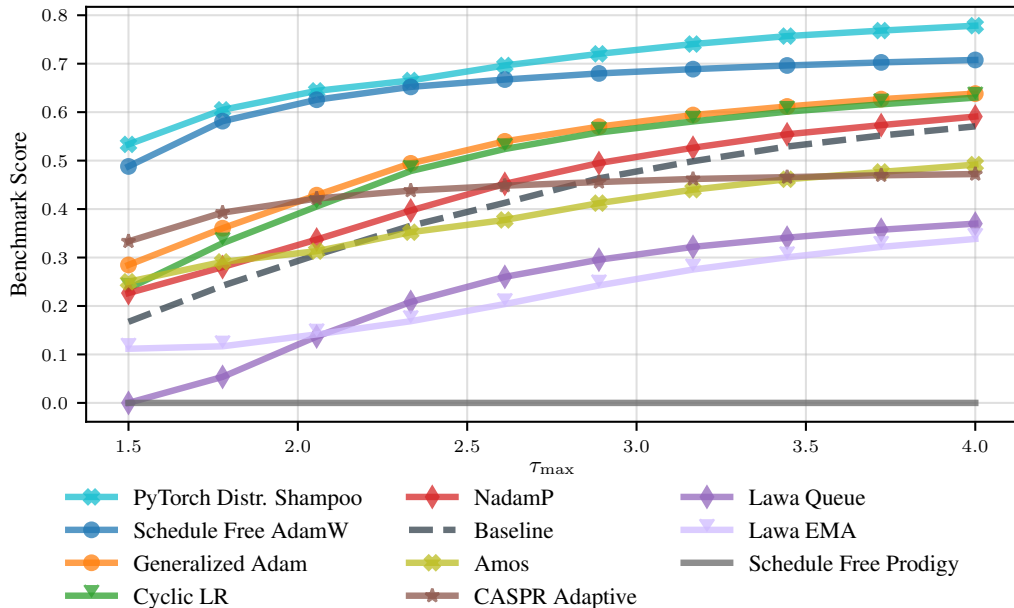
Submission	Line	Score
PYTORCH DISTRIBUTED SHAMPOO		0.7784
SCHEDULE FREE ADAMW		0.7077
GENERALIZED ADAM		0.6383
CYCLIC LR		0.6301
NADAMP		0.5909
BASELINE		0.5707
AMOS		0.4918
CASPR ADAPTIVE		0.4722
LAWA QUEUE		0.3699
LAWA EMA		0.3384
SCHEDULE FREE PRODIGY		0

(a) External tuning leaderboard



(b) External tuning performance profiles

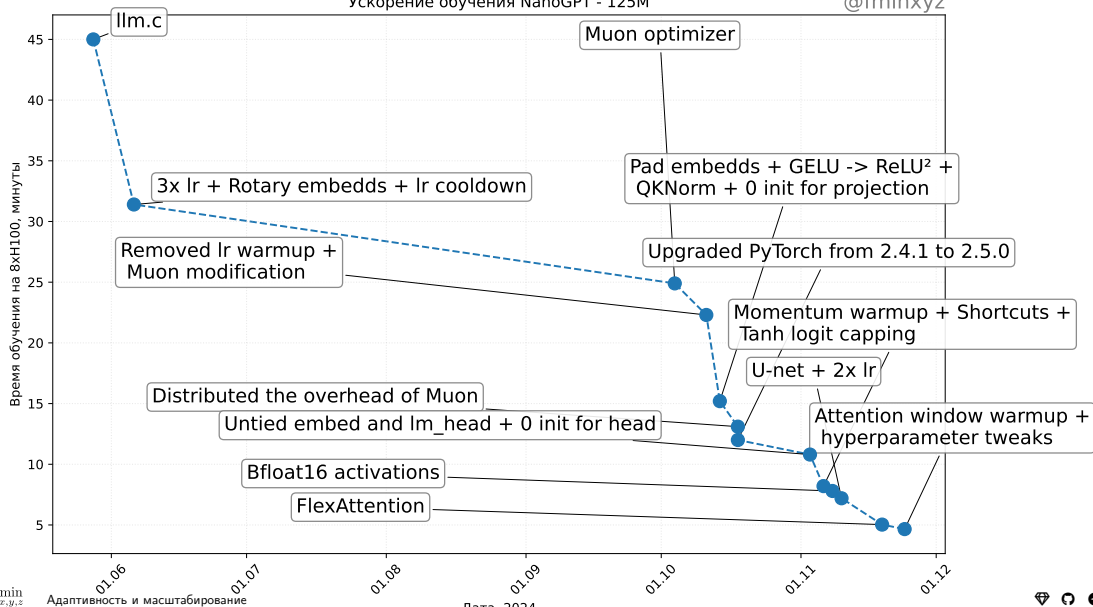
AlgoPerf benchmark



NanoGPT speedrun

Ускорение обучения NanoGPT - 125M

@fminxyz



Shampoo и Muon

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .

Замечания:

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.

Замечания:

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$ (обратный матричный корень).

Замечания:

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$ (обратный матричный корень).
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$ (обратный матричный корень).
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Учитывает информацию о кривизне эффективнее методов первого порядка.

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$ (обратный матричный корень).
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Учитывает информацию о кривизне эффективнее методов первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее по числу шагов.

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020) ³

Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks. Метод, вдохновлённый оптимизацией второго порядка.

Основная идея: Приближает полноматричный предобусловитель AdaGrad через произведения Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta)G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta)G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$ (обратный матричный корень).
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Учитывает информацию о кривизне эффективнее методов первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее по числу шагов.
- Требуется аккуратной реализации (эффективное вычисление обратных матричных корней).

³Gupta, V., Koren, T. and Singer, Y., 2018. Shampoo: Preconditioned stochastic tensor optimization. ICML.

Муон: вывод через наикрутейший спуск в спектральной норме ⁴ ⁵

Задача наикрутейшего спуска для матричного параметра W :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где $\|\cdot\|_\sigma$ — спектральная норма (максимальное сингулярное число).

⁴J. Bernstein "Deriving Muon". 2025.

⁵Kovalev, D. (2025). Understanding Gradient Orthogonalization.

Муон: вывод через наикрутейший спуск в спектральной норме ⁴ ⁵

Задача наикрутейшего спуска для матричного параметра W :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где $\|\cdot\|_\sigma$ — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса): $D^* = -G/\|G\|_F$ — направление $\propto$ градиенту

⁴J. Bernstein "Deriving Muon". 2025.

⁵Kovalev, D. (2025). Understanding Gradient Orthogonalization.

Муон: вывод через наикрутейший спуск в спектральной норме ⁴ ⁵

Задача наикрутейшего спуска для матричного параметра W :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где $\|\cdot\|_\sigma$ — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса): $D^* = -G/\|G\|_F$ — направление $\propto$ градиенту
- **SignGD** (ℓ_∞ -норма): $D^* = -\text{sign}(G)$ — знак каждого элемента

⁴J. Bernstein "Deriving Muon". 2025.

⁵Kovalev, D. (2025). Understanding Gradient Orthogonalization.

Муон: вывод через наикрутейший спуск в спектральной норме ⁴ ⁵

Задача наикрутейшего спуска для матричного параметра W :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где $\|\cdot\|_\sigma$ — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса): $D^* = -G/\|G\|_F$ — направление $\propto$ градиенту
- **SignGD** (ℓ_∞ -норма): $D^* = -\text{sign}(G)$ — знак каждого элемента
- **Muon** (спектральная норма): $D^* = -UV^\top$ — полярный фактор градиента

⁴J. Bernstein "Deriving Muon". 2025.

⁵Kovalev, D. (2025). Understanding Gradient Orthogonalization.

Muon: вывод через наикрутейший спуск в спектральной норме ⁴ ⁵

Задача наикрутейшего спуска для матричного параметра W :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где $\|\cdot\|_\sigma$ — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса): $D^* = -G/\|G\|_F$ — направление $\propto$ градиенту
- **SignGD** (ℓ_∞ -норма): $D^* = -\text{sign}(G)$ — знак каждого элемента
- **Muon** (спектральная норма): $D^* = -UV^\top$ — полярный фактор градиента

⁴J. Bernstein "Deriving Muon". 2025.

⁵Kovalev, D. (2025). Understanding Gradient Orthogonalization.

Muon: вывод через наикрутейший спуск в спектральной норме ⁴ ⁵

Задача наикрутейшего спуска для матричного параметра W :

$$D^* = \arg \min_{\|D\|_\sigma \leq 1} \text{tr} [\nabla L(W)^\top D]$$

где $\|\cdot\|_\sigma$ — спектральная норма (максимальное сингулярное число).

- **SGD** (норма Фробениуса): $D^* = -G/\|G\|_F$ — направление $\propto$ градиенту
- **SignGD** (ℓ_∞ -норма): $D^* = -\text{sign}(G)$ — знак каждого элемента
- **Muon** (спектральная норма): $D^* = -UV^\top$ — полярный фактор градиента

Двойственная норма к спектральной — **ядерная** (сумма сингулярных чисел):

$$\|G\|_* = \sum_i \sigma_i(G), \quad \arg \min_{\|D\|_\sigma \leq 1} \langle G, D \rangle = -UV^\top$$

⁴J. Bernstein "Deriving Muon". 2025.

⁵Kovalev, D. (2025). Understanding Gradient Orthogonalization.

Муон: упрощение Shampoo ⁶

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (US^2U^\top)^{-1/4} (USV^\top)(VS^2V^\top)^{-1/4} \\ &= W_t - \eta (US^{-1/2}U^\top)(USV^\top)(VS^{-1/2}V^\top) \\ &= W_t - \eta US^{-1/2}SS^{-1/2}V^\top \\ &= W_t - \eta UV^\top \end{aligned}$$

⁶Deriving Muon

Муон: упрощение Shampoo ⁶

$$\begin{aligned} W_{t+1} &= W_t - \eta \underbrace{(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}}_{\text{Shampoo}} \\ &= W_t - \eta (US^2U^\top)^{-1/4} (USV^\top) (VS^2V^\top)^{-1/4} \\ &= W_t - \eta (US^{-1/2}U^\top) (USV^\top) (VS^{-1/2}V^\top) \\ &= W_t - \eta US^{-1/2}SS^{-1/2}V^\top \\ &= W_t - \eta UV^\top \end{aligned}$$

Ключевой инсайт: Shampoo с «идеальной» статистикой (один градиент вместо накопленных) — это просто ортогонализация градиента. Все сингулярные числа обновления равны 1.

⁶Deriving Muon

Algorithm 1 Proposed optimizer

Require: Learning rate η , momentum μ , weight decay λ

- 1: Initialize $B_0 \leftarrow 0$
 - 2: **for** $t = 1, \dots$ **do**
 - 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
 - 4: $B_t \leftarrow \mu B_{t-1} + G_t$
 - 5: $G_t \leftarrow \mu B_t + G_t$
 - 6: $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
 - 7: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta(O_t + \lambda\theta_{t-1})$
 - 8: **end for**
 - 9: **return** θ_t
-

Newton-Schulz вместо SVD для вычисления $UV^{\top}$:

$$X_0 = G / \|G\|_F$$

$$X_{k+1} = aX_k + bX_kX_k^{\top}X_k + c(X_kX_k^{\top})^2X_k$$

с коэффициентами $a=3.4445$, $b=-4.7750$,
 $c=2.0315$.

- **5 итераций** достаточно на практике

Algorithm 1 Proposed optimizer

Require: Learning rate η , momentum μ , weight decay λ

- 1: Initialize $B_0 \leftarrow 0$
 - 2: **for** $t = 1, \dots$ **do**
 - 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
 - 4: $B_t \leftarrow \mu B_{t-1} + G_t$
 - 5: $G_t \leftarrow \mu B_t + G_t$
 - 6: $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
 - 7: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta(O_t + \lambda \theta_{t-1})$
 - 8: **end for**
 - 9: **return** θ_t
-

Newton-Schulz вместо SVD для вычисления $UV^{\top}$:

$$X_0 = G / \|G\|_F$$

$$X_{k+1} = aX_k + bX_kX_k^{\top}X_k + c(X_kX_k^{\top})^2X_k$$

с коэффициентами $a=3.4445$, $b=-4.7750$,
 $c=2.0315$.

- **5 итераций** достаточно на практике
- Только матричные умножения — эффективно на GPU

Algorithm 1 Proposed optimizer

Require: Learning rate η , momentum μ , weight decay λ

- 1: Initialize $B_0 \leftarrow 0$
 - 2: **for** $t = 1, \dots$ **do**
 - 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
 - 4: $B_t \leftarrow \mu B_{t-1} + G_t$
 - 5: $G_t \leftarrow \mu B_t + G_t$
 - 6: $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
 - 7: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta(O_t + \lambda \theta_{t-1})$
 - 8: **end for**
 - 9: **return** θ_t
-

Newton-Schulz вместо SVD для вычисления $UV^{\top}$:

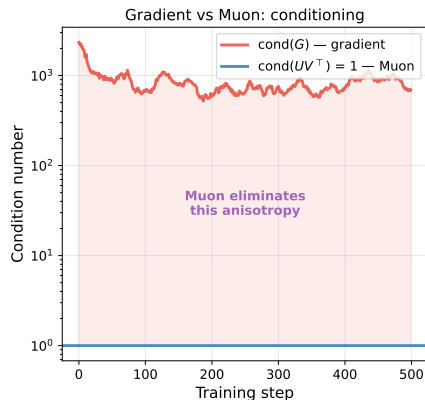
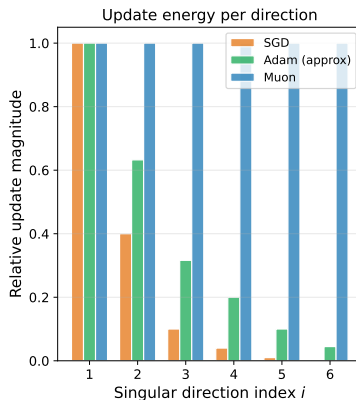
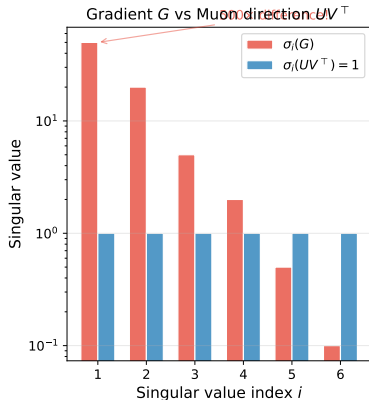
$$X_0 = G / \|G\|_F$$

$$X_{k+1} = aX_k + bX_kX_k^{\top}X_k + c(X_kX_k^{\top})^2X_k$$

с коэффициентами $a=3.4445$, $b=-4.7750$,
 $c=2.0315$.

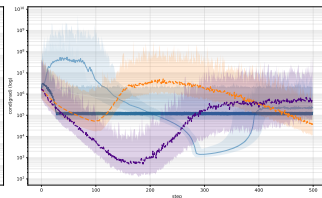
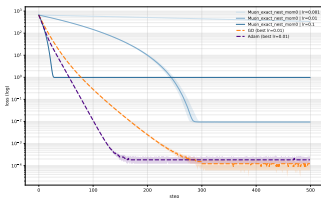
- **5 итераций** достаточно на практике
- Только матричные умножения — эффективно на GPU
- Кубическая сходимость к полярному фактору

Муон: геометрия ортогонализации



Вывод: SGD концентрирует обновление на доминирующих сингулярных направлениях (разница 500×). Муон нормализует все направления к единице — **равномерный прогресс** во всех направлениях пространства параметров.

Муон на квадратичных задачах ⁷

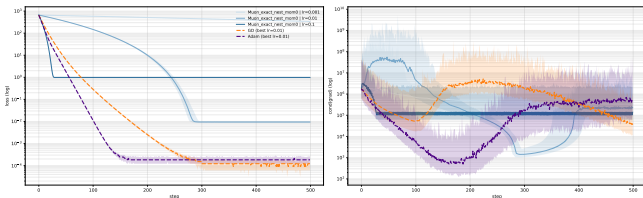


Квадратичная задача $L(W) = \frac{1}{2}\|W\|_F^2$ с гауссовским спектром собственных значений.

- Муон с $\eta = 0.01$ сходится **на порядки быстрее** GD и Adam

⁷Gonon, A., Muşat, A.-A., Boumal, N. (2026). Insights on Muon from Simple Quadratics.

Муон на квадратичных задачах ⁷

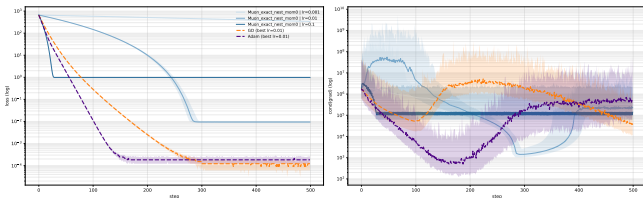


Квадратичная задача $L(W) = \frac{1}{2}\|W\|_F^2$ с гауссовским спектром собственных значений.

- Муон с $\eta = 0.01$ сходится **на порядки быстрее** GD и Adam
- Муон с $\eta = 0.1$ расходится — чувствительность к lr!

⁷Gonon, A., Muşat, A.-A., Boumal, N. (2026). Insights on Muon from Simple Quadratics.

Муон на квадратичных задачах ⁷

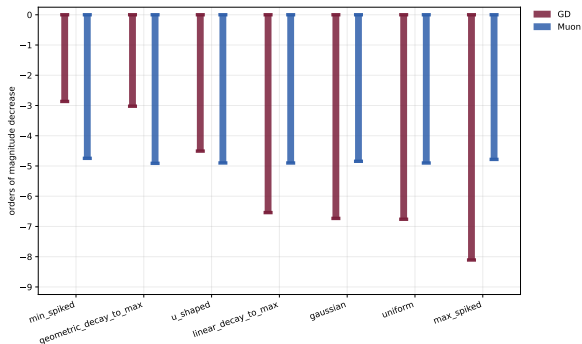


Квадратичная задача $L(W) = \frac{1}{2}\|W\|_F^2$ с гауссовским спектром собственных значений.

- Муон с $\eta = 0.01$ сходится **на порядки быстрее** GD и Adam
- Муон с $\eta = 0.1$ расходится — чувствительность к lr!
- Правая панель: число обусловленности градиента $\sim 10^{10}$ — Муон это **игнорирует**

⁷Gonon, A., Muşat, A.-A., Boumal, N. (2026). Insights on Muon from Simple Quadratics.

Муон: стабильность по спектру задачи⁸

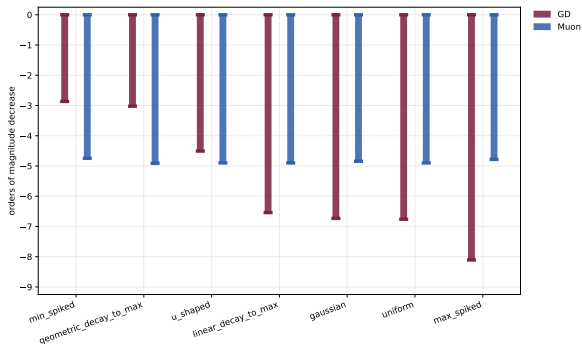


7 типов распределения собственных значений (от *max_spiked* до *uniform*).

- **GD**: от 3 до 8 порядков — **сильно зависит** от спектра

⁸Gonon, Muşat, Boumal (2026)

Муон: стабильность по спектру задачи⁸

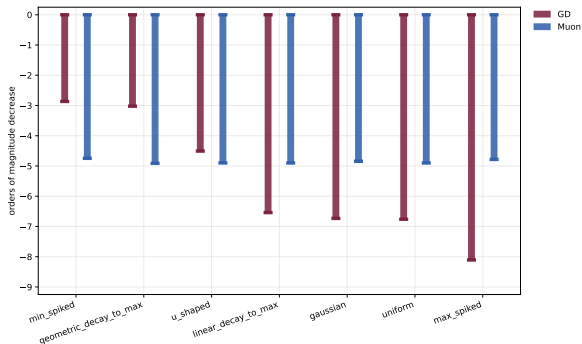


7 типов распределения собственных значений (от *max_spiked* до *uniform*).

- **GD**: от 3 до 8 порядков — **сильно зависит** от спектра
- **Муон**: стабильно ≈ 5 порядков — **инвариантен** к структуре спектра

⁸Gonon, Muşat, Boumal (2026)

Муон: стабильность по спектру задачи⁸

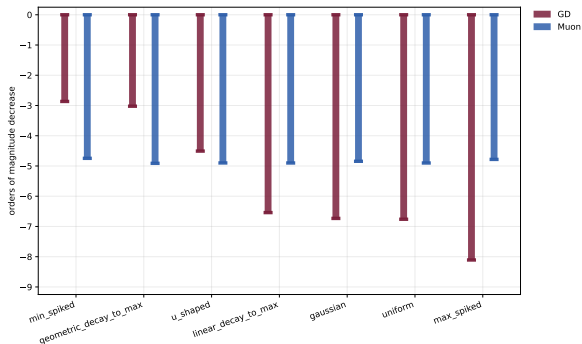


7 типов распределения собственных значений (от *max_spiked* до *uniform*).

- **GD**: от 3 до 8 порядков — **сильно зависит** от спектра
- **Муон**: стабильно ≈ 5 порядков — **инвариантен** к структуре спектра

⁸Gonon, Muşat, Boumal (2026)

Муон: стабильность по спектру задачи⁸



7 типов распределения собственных значений (от *max_spiked* до *uniform*).

- **GD**: от 3 до 8 порядков — **сильно зависит** от спектра
- **Муон**: стабильно ≈ 5 порядков — **инвариантен** к структуре спектра

Вывод: ортогонализация градиента устраняет зависимость от числа обусловленности задачи. Муон делает одинаковый прогресс независимо от структуры Гессиана.

⁸Gonon, Muşat, Boumal (2026)

Мagma: случайное маскирование обновлений⁹

Идея: случайное пропускание обновлений параметров во время обучения LLM **улучшает** результат!

SkipUpdate: на каждом шаге блокируем обновление блока параметров с вероятностью $1 - p$, масштабируя оставшиеся на $1/p$.

⁹Joo, T. et al. (2026). On Surprising Effectiveness of Masking Updates in Adaptive Optimizers.

Magma: случайное маскирование обновлений⁹

Идея: случайное пропускание обновлений параметров во время обучения LLM **улучшает** результат!

SkipUpdate: на каждом шаге блокируем обновление блока параметров с вероятностью $1 - p$, масштабируя оставшиеся на $1/p$.

Magma (Momentum-Aligned Gradient Masking): модулирует маскирование через выравнивание с моментумом:

$$\tilde{s}_t^{(b)} = \text{sigmoid} \left(\frac{\cos(\mu_t^{(b)}, g_t^{(b)})}{\tau} \right)$$

⁹Joo, T. et al. (2026). On Surprising Effectiveness of Masking Updates in Adaptive Optimizers.

Magma: случайное маскирование обновлений⁹

Идея: случайное пропускание обновлений параметров во время обучения LLM **улучшает** результат!

SkipUpdate: на каждом шаге блокируем обновление блока параметров с вероятностью $1 - p$, масштабируя оставшиеся на $1/p$.

Magma (Momentum-Aligned Gradient Masking): модулирует маскирование через выравнивание с моментумом:

$$\tilde{s}_t^{(b)} = \text{sigmoid} \left(\frac{\cos(\mu_t^{(b)}, g_t^{(b)})}{\tau} \right)$$

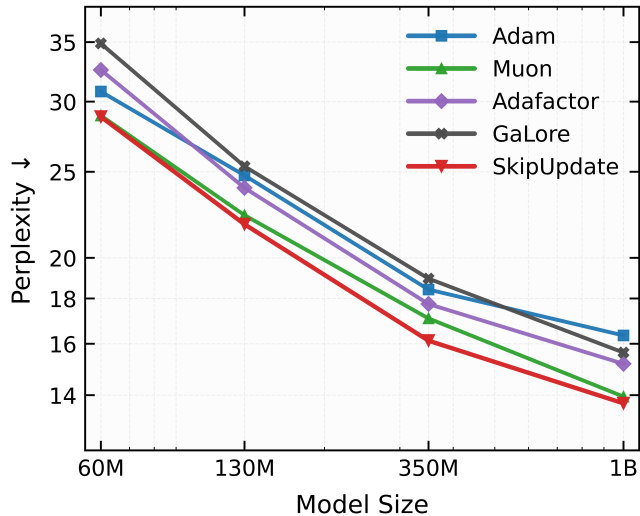
Теоретический инсайт: маскирование индуцирует геометрическую регуляризацию, зависящую от кривизны:

$$\mathcal{R}_t^{(b)} = \frac{1-p}{2p} \cdot (\Delta_t^{(b)})^\top H_{bb}(\theta_t) \Delta_t^{(b)}$$

Штрафует обновления вдоль «острых» направлений $\rightarrow$ оптимизация смещается к более плоским минимумам.

⁹Joo, T. et al. (2026). On Surprising Effectiveness of Masking Updates in Adaptive Optimizers.

Магма: масштабирование до 1B параметров¹⁰



Вывод: SkipUpdate+Damping (Magma) стабильно превосходит Adam, Muon, Adafactor и GaLore на всех масштабах от 60M до 1B. На 1B параметрах: perplexity 13.8 vs 16.3 (Adam) — улучшение на **19%**.

Дополнительные материалы

-  J. Bernstein "Deriving Muon"

Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"

Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)

Дополнительные материалы

-  J. Bernstein "Deriving Muon"
-  K. Jordan "Muon: An optimizer for hidden layers"
-  Gonon et al. "Insights on Muon from Simple Quadratics" (2026)
-  Joo et al. "On Surprising Effectiveness of Masking Updates" (2026)

Дополнительные материалы

- 🔗 J. Bernstein "Deriving Muon"
- 🔗 K. Jordan "Muon: An optimizer for hidden layers"
- 🔗 Gonon et al. "Insights on Muon from Simple Quadratics" (2026)
- 🔗 Joo et al. "On Surprising Effectiveness of Masking Updates" (2026)
- 🎥 Д. Ковалёв "Understanding Gradient Orthogonalization" (теория Muon)